

23 Regular Grammars and Automata

23.1 Grammars to Automata

Exercise 23.1. Construct a finite state automaton that recognizes the language generated by the following regular grammar.

- terminals: 0 and 1
- nonterminals: S and A
- start symbol: S .
- production rules: $S \rightarrow 1A$, $S \rightarrow 0$, $S \rightarrow \lambda$, $A \rightarrow 0A$, $A \rightarrow 1A$, $A \rightarrow 1$

Exercise 23.2. Construct a finite state automaton that recognizes the language generated by the following regular grammar.

- terminals: 0 and 1
- nonterminals: S , A , B
- start symbol: S .
- production rules: $S \rightarrow 1A$, $S \rightarrow 0$, $A \rightarrow 0B$, $A \rightarrow 1A$, $A \rightarrow 1$, $B \rightarrow 0A$, $B \rightarrow 1B$, $B \rightarrow 0$

Exercise 23.3. Explain how *any* regular grammar G can be converted into an NFA N such that $L(N) = L(G)$.

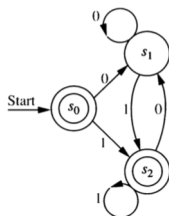
- What is a regular grammar? (What sorts of production rules are allowed?)
- What will the states of N be? How many will there be?
- What state will be the start state?
- Which states will be final (ie. accepting) states?
- How does each rule in G get converted into a part of N ? [Go through each kind of rule a regular grammar may have.]

23.2 Automata to Grammars

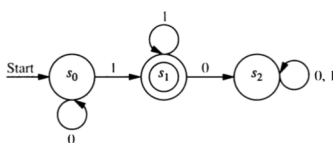
Exercise 23.4. Show that every DFA or NFA is equivalent to a DFA or NFA with a “lonely start state”. A lonely start state is a start state that can never be returned to. (In terms of the graph, its in-degree is 0.)

Exercise 23.5. Convert each of the automata in the next three problems into an automaton with a lonely start state.

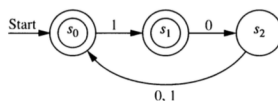
Exercise 23.6. Construct a regular grammar that generates the language accepted by this finite state automaton.



Exercise 23.7. Construct a regular grammar that generates the language accepted by this finite state automaton.



Exercise 23.8. Construct a regular grammar that generates the language accepted by this finite state automaton.



Exercise 23.9. In this problem you will show that any DFA or NFA with a lonely start state is equivalent to a regular grammar by showing how to construct a grammar G from such an automaton.

- What will the terminals and nonterminals in your grammar be?
- What will the start symbol be?
- How are the production rules created?
- Why was it necessary to have a lonely start state?

23.3 Some Review

Exercise 23.10. What does it mean to say that two machines/grammars/regular expressions are equivalent?

Exercise 23.11. Convert each of the following NFAs into an equivalent DFA.
Do this using our general algorithm for this task. For the first two, the start state is S .

